

Ros Day02 hw1 보고서

작성자: Doyeon Park

작성일: 2026.09.10

목차

1. 과제 개요
2. 커스텀 메시지 정의
3. 패키지 및 통신 구조
4. Publisher 구현
5. Subscriber 구현
6. 실행 결과

1. 과제 개요

이번 과제는 ROS2 에서 커스텀 메시지를 직접 만들고, 해당 메시지를 이용하여 Publisher 와 Subscriber 사이에서 Topic 통신을 구현하는 것이었다.

Publisher 에서는 터미널로 여러 개의 정수를 입력받고, 입력받은 정수들은 C++의 vector 에 저장한 뒤 커스텀 메시지에 넣어 발행하였다. Subscriber 에서는 메시지를 수신하고 벡터에 들어 있는 값을 터미널에 출력하도록 구현하였다.

2. 커스텀 메시지 정의

커스텀 메시지를 관리하기 위해 다음 패키지를 생성하였다.

```
parkdoyeon_day2_interfaces
```

패키지 내부에 msg 폴더를 만들고 IntVector.msg 파일을 생성하였다. 메시지 내용은 다음과 같다.

```
int32[] data
```

int32 는 32 비트 정수 자료형을 의미하고, 대괄호는 여러 개의 데이터를 저장할 수 있는 배열을 의미한다. 배열의 길이를 따로 지정하지 않았기 때문에 입력하는 정수의 개수에 따라 메시지의 길이가 달라질 수 있다.

C++에서 해당 data 필드는 std::vector 와 같은 형태로 사용할 수 있다. 따라서 터미널에서 입력받은 정수들을 vector 에 저장한 뒤 메시지의 data 에 대입할 수 있다.

커스텀 메시지가 생성되도록 CMakeLists.txt 에는 rosidl_default_generators 와 rosidl_generate_interfaces 설정을 추가하였다. package.xml 에는 메시지 생성과 실행에 필요한 rosidl_default_generators, rosidl_default_runtime 의존성을 추가하였다.



```
int32[] data
doyeon@doyeon-park:~/colcon_ws$
```

ros2 interface show 명령을 실행한 결과 int32[] data 가 출력되는 것을 볼 수있다. 이를 통해 IntVector 메시지가 ROS2 에 정상적으로 등록된 것을 확인하였다.

3. 패키지 및 통신 구조

이번 과제에서는 메시지 정의와 통신 노드를 분리하기 위해 두 개의 패키지를 사용하였다.

```
parkdoyeon_day2_interfaces
```

커스텀 메시지인 IntVector.msg 를 정의하는 패키지이다.

```
parkdoyeon_day2_hw1
```

Publisher 와 Subscriber 노드를 실행하는 C++ 패키지이다.

통신 과정은 다음과 같다.

터미널 정수 입력

→ std::vector 에 데이터 저장

- IntVector 메시지의 data 에 대입
- vector_topic 으로 발행
- Subscriber 가 메시지 수신
- 수신한 벡터를 터미널에 출력

Publisher 와 Subscriber 는 모두 vector_topic 을 사용하였다. 두 노드에서 Topic 이름과 메시지 자료형이 같아야 정상적으로 통신할 수 있다.

소스 파일과 헤더 파일을 분리하기 위해 include 폴더에는 클래스 선언을 작성하고, src 폴더에는 클래스의 실제 동작과 main 함수를 작성하였다.

Publisher 관련 파일

- vector_publisher.hpp
- vector_publisher.cpp
- publisher_main.cpp

Subscriber 관련 파일

- vector_subscriber.hpp
- vector_subscriber.cpp
- subscriber_main.cpp

4. Publisher 구현

Publisher 는 vector_publisher 라는 이름의 ROS2 노드로 생성하였다. IntVector 형식의 메시지를 vector_topic 으로 발행하도록 Publisher 를 설정하였다.

터미널에서 한 줄의 문자열을 입력받고, std::istringstream 을 사용하여 공백을 기준으로 정수를 분리하였다. 분리한 값은 다음 코드를 이용하여 vector 에 저장하였다.

```
std::vector<std::int32_t> values;

while (input >> value) {
    values.push_back(value);
}
```

예를 들어 터미널에서 다음과 같이 입력한다.

10 20 -3

입력값은 순서대로 values 벡터에 저장된다.

values = [10, 20, -3]

입력한 내용에 정수가 아닌 값이 포함되어 있으면 메시지를 발행하지 않고 “정수만 입력해주세요.”라는 안내를 출력하도록 하였다. 아무 값도 입력하지 않은 경우에도 빈 메시지를 발행하지 않도록 처리하였다.

vector 에 값이 저장되면 publish_vector 함수를 호출한다.

```
node->publish_vector(values);
```

publish_vector 함수에서는 커스텀 메시지 객체를 생성하고 vector 데이터를 메시지에 저장한 뒤 발행한다.

```
message.data = values;
publisher_ ->publish(message);
```

메시지를 발행한 후 실제로 전송한 값을 확인할 수 있도록 Publisher 터미널에도 벡터 내용을 출력하였다.

5. Subscriber 구현

Subscriber 는 vector_subscriber 라는 이름의 ROS2 노드로 생성하였다. Publisher 와 동일한 vector_topic 과 IntVector 메시지 형식을 사용하였다.

Subscriber 는 create_subscription 함수를 이용하여 vector_topic 을 구독한다. 메시지가 도착하면 vector_callback 함수가 자동으로 실행되도록 연결하였다.

vector_callback 함수에서는 수신한 message 의 data 를 처음부터 끝까지 반복하면서 값을 읽는다.

```
for (std::size_t i = 0; i < message->data.size(); ++i) {
    output << message->data[i];
}
```

각 숫자 사이에는 쉼표를 넣어 벡터 형태를 알아보기 쉽게 출력하였다.

Subscriber 의 main 함수에서는 다음 함수를 사용하였다.

```
rclcpp::spin(node);
```

rclcpp::spin 은 Subscriber 노드가 종료되지 않고 계속 실행되면서 새로운 메시지가 도착하는지 기다리도록 한다.

6. 실행 결과

먼저 colcon build 명령으로 커스텀 메시지 패키지와 C++ 통신 패키지를 빌드하였다.

```
colcon build --packages-select parkdoyeon_day2_interfaces parkdoyeon_day2_hw1
```

빌드 결과 두 패키지가 모두 정상적으로 완료되었다.

Subscriber 를 먼저 실행하여 vector_topic 의 메시지를 기다리게 하였다.

```
ros2 run parkdoyeon_day2_hw1 vector_subscriber
```

다른 터미널에서 Publisher 를 실행하였다.

```
ros2 run parkdoyeon_day2_hw1 vector_publisher
```

Publisher 에 여러 형태의 정수 데이터를 입력하여 통신을 확인하였다.

입력 1

```
10 10 10
```

Publisher 출력

```
Published: [10, 10, 10]
```

Subscriber 출력

Received: [10, 10, 10]

입력 2

1 2 3

Subscriber 출력

Received: [1, 2, 3]

입력 3

10 20 30 40 50

Subscriber 출력

Received: [10, 20, 30, 40, 50]

입력 4

-10 0 10

Subscriber 출력

Received: [-10, 0, 10]

입력한 벡터와 Subscriber에서 출력된 벡터의 값과 순서가 모두 일치하였다. 또한 원소가 3개인 벡터와 5개인 벡터가 모두 정상적으로 전달되었다. 이를 통해 IntVector 메시지의 배열 길이가 입력값에 따라 변경되는 것을 확인하였다. 음수가 포함된 메시지도 정상적으로 통신하였다.

```
doyeon@doyeon-park:~$ cd ~/colcon_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run parkdoyeon_day2_hw1 vector_publisher
공백으로 구분한 정수를 입력하세요.
> 10 20 -3
공백이 아니면 q를 입력하세요.
> 10 20 -3
[INFO] [1788954363.161375029] [vector_publisher]: Published: [10, 20, -3]
> 10 10
[INFO] [1788954376.207629573] [vector_publisher]: Published: [10, 10, 10]
> 1 2 3
[INFO] [1788954477.521500321] [vector_publisher]: Published: [1, 2, 3]
> 10 20 30 40 50
[INFO] [1788954483.629125925] [vector_publisher]: Published: [10, 20, 30, 40, 50]
> -10 0 10
[INFO] [1788954486.509071339] [vector_publisher]: Published: [-10, 0, 10]
^C

doyeon@doyeon-park:~$ cd ~/colcon_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run parkdoyeon_day2_hw1 vector_subscriber
No executable found
doyeon@doyeon-park:~$ cd ~/colcon_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run parkdoyeon_day2_hw1 vector_subscriber
No executable found
doyeon@doyeon-park:~$ cd ~/colcon_ws
source /opt/ros/jazzy/setup.bash
colcon build --packages-select parkdoyeon_day2_hw1
source install/setup.bash
ros2 pkg executables parkdoyeon_day2_hw1
Starting >>> parkdoyeon_day2_hw1
Finished <<< parkdoyeon_day2_hw1 [5.78s]

Summary: 1 package finished [5.88s]
parkdoyeon_day2_hw1 vector_publisher
parkdoyeon_day2_hw1 vector_subscriber
doyeon@doyeon-park:~$ cd ~/colcon_ws$ ros2 run parkdoyeon_day2_hw1 vector_subscriber
[INFO] [1788954342.786722993] [vector_subscriber]: Waiting for messages on vector_topic...
[INFO] [1788954363.161703040] [vector_subscriber]: Received: [10, 20, -3]
[INFO] [1788954376.207874190] [vector_subscriber]: Received: [10, 10, 10]
[INFO] [1788954477.521900532] [vector_subscriber]: Received: [1, 2, 3]
[INFO] [1788954483.621256726] [vector_subscriber]: Received: [10, 20, 30, 40, 50]
[INFO] [1788954486.509314335] [vector_subscriber]: Received: [-10, 0, 10]
^C
```